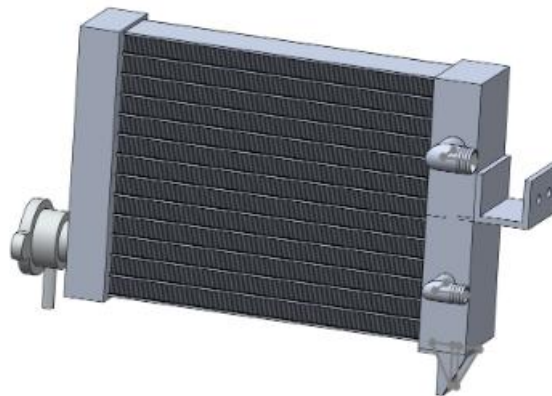


DESIGN REPORT:

Radiator Thermal Design



ϵ -NTU Modelling & Transient Validation for the EV Powertrain Cooling System

Darsh Raghuvanshi

Drivetrain Subdivision

Junior Design Engineer - IIT Bombay Racing



Contents

1. Introduction and Overview	3
1.1 GitHub Repository.....	3
1.2 Physics Background: The ϵ -NTU Method.....	4
1.3 Notation (Kays & London Convention)	4
2. Steady-State Design Script (radiator_steady.mlx).....	5
2.1 Required Heat Rejection.....	5
2.2 Coolant (Water) Properties via CoolProp.....	5
2.3 Scrapped Methodology.....	6
2.4 Fixed Geometry & Sweep Variables	7
2.5 Water-Side Convection Coefficient.....	7
2.6 Air-Side Convection Coefficient	7
2.7 Overall Conductance and Capacity Rates.....	8
2.8 Fin Efficiency / Effective Area and Fin Effectiveness	8
2.9 NTU, ϵ , and Predicted Heat Rejection.....	10
2.10 Extracting the Design Point.....	10
2.11 Water-Side Pressure Drop	11
2.12 Sensitivity Studies	12
2.13 A_face vs. FPI Summary Table.....	13
3. Transient Models (radiator_transient_fmot.mlx / _emrax.mlx)	14
3.1 Extracting Data as Input	14
3.2 Data Cleaning: Zero-RPM Filtering & Time Shifting.....	14
3.3 Motor + Controller Efficiency Map.....	15
3.4 Initial Parameters (Flagged for Iteration)	16
3.5 Deriving Air Velocity from Wheel Kinematics.....	16
3.6 Q_epsilonNTU Function	16
3.7 Time-Marching Loop (Lumped Capacitance, Explicit Euler)	17
3.8 Outputs	17
4. F-MOT vs. EMRAX: What's Different	18
5. Validation Suite (radiator_transient_test.mlx)	18
5.1 Independent Steady-State Solver.....	18
5.2 Test Case Matrix.....	19
5.3 Time Constant and t99.....	19
5.4 Summary Table and Pass/Fail Checks.....	20
6. Known Assumptions and Simplifications	20
7. Suggested Workflow	21

File Contents

This report documents the thermal sizing and validation of the radiator used to cool the 2WD powertrain (one motor and one motor controller per side). The work is split into a steady-state design/sizing tool and a transient (time-marching) validation tool, run against real endurance-run data for the outgoing motor, plus a synthetic test suite that validates the transient solver before its tested with real test-run data.

File	Purpose
<code>radiator_steady.mlx</code>	Core sizing tool. Sweeps FPI and frontal area to find the minimum radiator geometry that rejects the target heat load, using the ϵ -NTU method.
<code>radiator_transient_fmot.mlx</code>	Calculates coolant temperature over a real endurance-run data set for the F-MOT motor, using the geometry chosen in the steady script.
<code>radiator_transient_emrax.mlx</code>	Identical structure to the F-MOT script, but with the EMRAX motor + controller efficiency map substituted in.
<code>radiator_transient_test.mlx</code>	Constant-condition test cases that validate the transient model against an independently solved steady-state temperature, before the model is trusted against actual data.

Each transient script has its own copy of the `Q_epsilonNTU` function, the ϵ -NTU heat-rejection calculation (identical to the steady-state script), so it can be called once per timestep.

1. Introduction and Overview

This file documents the methodology, assumptions, and current state of the modelling toolchain, and is intended to be read alongside the MATLAB scripts.

The toolchain is split into two complementary tools:

- A **steady-state design / sizing tool** (`radiator_steady.mlx`) that sweeps fin density and frontal area to find the minimum radiator geometry that clears a fixed, conservative worst-case heat load.
- A **transient (time-marching) validation tool** (`radiator_transient_fmot.mlx` / `radiator_transient_emrax.mlx`) that replays real endurance-run data through the chosen geometry to optimize the design (as steady state assumes worst-case conditions, radiator will be over optimized initially) for the actual accel / brake / idle profile of an event.

A **synthetic validation suite** (`radiator_transient_test.mlx`) tests the transient solver against constant-condition cases with an independently-solved steady-state answer, so the transient solver implementation is trusted before it is run against real data.

1.1 GitHub Repository

This document is meant to be read along with all the codes which can be found in the attached GitHub Repository.

https://github.com/darshysys/Radiator_design

1.2 Physics Background: The ϵ -NTU Method

Sizing a radiator means finding the heat rejected, Q , for a given geometry and a given set of operating conditions, without needing to iteratively guess an outlet temperature. In the steady state effectively all temperatures are known or assumed ($T_{\text{coolant_in}}$ fixed, $T_{\text{coolant_out}}$ capped due to limit on motor controller, T_{air} is ambient temp and $T_{\text{air_out}}$ can be calculated via energy balance on Q).

This selection only becomes apparent once the model moves to the transient state. At every timestep Q is taken as input and updates $T_{\text{coolant_out}}$ is calculated (here the motor controller is inapplicable as we are only running our model for a certain timestep not the entire temperature range possible. ϵ -NTU avoids this because ϵ is a function of NTU and C_r alone, both computable from known quantities (UA from geometry, $\dot{C}_{\min}/\dot{C}_{\max}$ from mass flow and c_p) without reference to any outlet temperature

Because the transient model requires ϵ -NTU on these grounds, the steady-state sizing model was also implemented in ϵ -NTU.

- **Effectiveness, ϵ :** the ratio of actual heat transfer to the maximum thermodynamically possible heat transfer (i.e. if one fluid stream were brought all the way to the other stream's inlet temperature).
- **NTU (Number of Transfer Units):** $NTU = UA / C_{\min}$, a non-dimensional measure of how large the heat exchanger is relative to the smaller of the two fluid capacity rates.
- **$C_{\min}$, $C_{\max}$, C^* :** the fluid capacity rates $C = \dot{m} \cdot c_p$ for each stream; $C_{\min}$ is the smaller of the two (air or water side), $C^* = C_{\min}/C_{\max}$.

Given ϵ , $Q = \epsilon \cdot C_{\min} \cdot (T_{\text{coolant_in}} - T_{\text{air_in}})$, with no need to solve for the outlet temperatures directly.

All scripts use the same closed-form correlation for a **crossflow heat exchanger with both fluids unmixed** (the standard configuration for a fin-and-tube radiator), taken from Kays & London:

$$\epsilon = 1 - \exp((NTU^{0.22}/C_{\text{star}}) * (\exp(-C_{\text{star}} * NTU^{0.78}) - 1));$$

Everything else in the scripts exists to compute the two inputs this formula needs: $U \cdot A$ (the overall thermal conductance) and $C_{\min}/C^*$.

1.3 Notation (Kays & London Convention)

The following symbol convention is used throughout all four scripts and this report:

Symbol	Meaning
$A_{\text{fr}} / A_{\text{face}}$	Frontal (face) area of the radiator core
A_o / A_{fins}	Effective, fin-efficiency-corrected air-side heat transfer area
σ (σ_{c})	Contraction ratio: ratio of minimum free-flow area to frontal area
G	Mass flux through the minimum free-flow area ($\dot{m}/A_{\min}$)

Symbol	Meaning
D_h	Hydraulic diameter of a flow passage
j	Colburn j-factor (heat transfer correlation)
f	Friction factor (pressure drop correlation)
NTU, ϵ , C_{min} , C_{max} , C^*	Standard heat-exchanger effectiveness terms

2. Steady-State Design Script (radiator_steady.mlx)

This is the primary sizing tool: for a grid of candidate frontal areas and fin densities, it computes predicted heat rejection and identifies the minimum geometry that clears the required heat load.

2.1 Required Heat Rejection

The motor's peak power and its worst-case (lowest) efficiency point from the datasheet are used to get a single conservative heat-loss number. The motor controller's dissipation is taken as a fixed worst-case figure rather than a function of load.

```
import py.CoolProp.CoolProp.*
P_peak = 35000; % peak operating power of motor (W), F-MOT motor datasheet
efficiency = 0.93; % worst efficiency according to datasheet (%)
Q_motor = P_peak - efficiency*P_peak; % heat loss through water (W)
Q_motor_controller = 750; % max power dissipation according to datasheet, (W)
Q_target = Q_motor + Q_motor_controller; % (W)
```

Flag: this fixed 750 W MC figure and the transient model's map-based combined motor+MC loss are two different accounting methods for the same physical dissipation.

2.2 Coolant (Water) Properties via CoolProp

Water properties are pulled from CoolProp (PropsSI, via MATLAB's Python interop) rather than hardcoded, so they can be re-evaluated at the correct bulk temperature. Density and specific heat are pulled once at a fixed reference temperature (50°C) since they barely change over the coolant's operating range. Once Q_{target} and $\dot{m}_{coolant}$ give a first-pass ΔT and average bulk temperature, viscosity, conductivity, and Prandtl number are re-evaluated at that average temperature, since these three properties matter more for the Reynolds/Nusselt calculation and are more temperature-sensitive.

Thermal conductivity (k_{water}), Prandtl number (Pr_{water}) and dynamic viscosity (μ_{water}) were chosen to be recalculated after temperature is fixed because they vary substantially within the operating temperature range (20°C-70°C):

Water Properties (20°C vs 70°C)			
Property	20°C	70°C	% Change
Thermal Cond. (k) [W/m-K]	0.598	0.663	+10.9%
Dyn. Viscosity (μ) [mPa-s]	1.002	0.404	-59.7%
Prandtl No. (Pr)	7.01	2.55	-63.6%

Figure 1: Variation of water properties across temperature range of 20°C-70°C

```
rho_water = PropsSI('D', 'T', 323.15, 'P', P_water, 'Water');
cp_water = PropsSI('C', 'T', 323.15, 'P', P_water, 'Water'); % using 323.15K

k_water = PropsSI('L', 'T', T_avg_K, 'P', P_water, 'Water'); % using T_avg_K
mu_water = PropsSI('V', 'T', T_avg_K, 'P', P_water, 'Water');
Pr_water = PropsSI('Prandtl', 'T', T_avg_K, 'P', P_water, 'Water');
```

Flag: rather than assuming 50°C, a more accurate bulk temperature value can be taken once the transient model is run on simulated data of the new motors (F-MOT).

2.3 Scrapped Methodology

On the first iteration of the code, rather than using f_{pi} as the sweep variable, fin density (σ , m^2/m^3) was swept directly as a measure of surface area added per unit volume occupied by the fins. This method was later scrapped in favour of computing fin efficiency, which needed concrete geometric numbers (fin height, fin thickness) rather than an arbitrary density value, and so that realistic numbers could be given to the manufacturer to get their opinion on manufacturing feasibility. Accordingly, this code snippet was commented out and not used in the final script:

```
% function [FPI] = sigma_to_fpi(sigma,f_height,f_thickness)
% p = 2*(f_height-f_thickness)/(sigma*f_height-2); %fin pitch (m)
% p_in = p/0.0254; %pitch (inches)
% FPI = 1/p_in;
% end
```

2.4 Fixed Geometry & Sweep Variables

Fin height (8 mm) and fin thickness (0.08 mm) are carried over from the senior team's reference script and held fixed - they are not being re-optimized in this iteration. The two free design variables being swept are `fpi_range` (fins per inch, 8 to 25) and `A_face_range` (radiator frontal area, 0.01 to 0.20 m²). Radiator height H is fixed at 0.35 m for the main sweep (its sensitivity is checked separately).

Flag: in the later phases of manufacturing, more accurate fin height and fin thickness numbers can be taken after talking with the manufacturer.

2.5 Water-Side Convection Coefficient (h_{water})

The coolant tube is treated as a rectangular duct. Its hydraulic diameter and cross-sectional area set up a standard internal-flow correlation:

```
a_tube = 0.002; % dimensions of rectangular tube, height as 2mm
b_tube = d; % width of tube taken equal to depth of radiator
tube_pitch = 0.010; % height of tube(2mm) + fin space(8mm), used to calculate
number of tubes
D_h_tube = (4*a_tube*b_tube)/(2*(a_tube+b_tube)); % hydraulic diameter
A_tube = a_tube*b_tube; % tube cross sectional area (m^2)
```

Flow regime is checked explicitly: laminar flow ($Re < 2300$) uses the constant fully-developed Nusselt number $Nu = 3.66$; turbulent flow uses the Dittus-Boelter correlation $Nu = 0.023 \cdot Re^{0.8} \cdot Pr^{0.3}$, $h_{\text{water}} = Nu \cdot k_{\text{water}} / D_{\text{h_tube}}$.

```
%water side convection heat transfer coefficient
n_tubes = floor(H/tube_pitch); %number of tubes
mdot_tube = mdot_coolant_kgs/n_tubes; %mass flow rate per tube
v_water = mdot_tube/(rho_water*A_tube); %velocity of water in each tube
Re_water = (rho_water*v_water*D_h_tube)/mu_water;
if Re_water < 2300
    Nu_water = 3.66; %laminar
else
    Nu_water = 0.023*(Re_water^0.8)*(Pr_water^0.3); %turbulent
end
h_water = (Nu_water*k_water)/D_h_tube;
```

2.6 Air-Side Convection Coefficient (h_{air})

The air side is where most of the fin geometry detail lives:

- **Contraction ratio (σ):** as air enters the fin channels between tubes, the flow area contracts. `sigma_c` captures this, and is used to convert the frontal-area-based approach velocity into the actual (higher) velocity through the minimum flow area.
- **Mass flux:** $G_{\text{air}} = \rho_{\text{air}} \cdot v_{\text{air}} / \sigma_c$. Captures the fact that when, for a constant mass flow rate, air travels faster through lesser area, so the frontal area disrupted by the tube and fins is taken into account using contraction ratio.

- **Fin channel hydraulic diameter:** from fin spacing ($0.0254/f_{pi}$) and fin height. 0.0254 is conversion factor from inches to metres.
- **Colburn j-factor:** $J_{air} = 0.174/(Re_{air})^{0.383}$, an empirical heat-transfer correlation for the fin geometry. This correlation is inherited from the prior code base and its original source/derivation has not yet been independently re-verified against the actual fin geometry class being used.

Flag: the Colburn j-factor value used here has not yet been independently confirmed against Kays & London's compact heat exchanger surface data for this fin geometry class.

- **Colburn analogy:** $h_{air} = J_{air} \cdot G_{air} \cdot c_{p_air} / Pr_{air}^{(2/3)}$ converts the j-factor into a convection coefficient.

```
%air side convection heat transfer coefficient
sigma_c = 1-((fpi*f_height*f_thickness)/0.0254+a_tube)/(a_tube+f_height);
%contraction factor
G_air = rho_air*v_air/sigma_c; % mass flux
f_spacing = 0.0254/fpi;
D_h_fin = (4*f_spacing*f_height)/(2*(f_spacing+f_height)); %hydraulic diameter of
fin channel
Re_air = G_air*D_h_fin/mu_air; %reynolds numbers
J_air = 0.174/(Re_air)^0.383; % Colburn j-factor
h_air = J_air*G_air*c_p_air/(Pr_air^0.667);
```

2.7 Overall Conductance and Capacity Rates

This treats the tube wall as having negligible thermal resistance, and contact/bond resistance between fin and tube as negligible, a simplification pending manufacturing/bonding-method confirmation.

```
U = 1/((1/h_air)+(1/h_water)); %overall heat transfer coefficient

%capacity rates
C_air = rho_air*v_air*c_p_air*A_face;
C_water = mdot_coolant_kgs*c_p_water;
C_min = min(C_air,C_water);
C_max = max(C_air,C_water);
C_star = C_min/C_max;
```

2.8 Fin Efficiency / Effective Area and Fin Effectiveness

There are 2 different heat transfer surfaces involved in the radiator: the bare tube wall in contact with air ($A_{primary}$) and the fin surface ($A_{secondary}$). Fins are not 100% efficient conductors, so a fin efficiency correction is applied.

Fin efficiency is evaluated using the standard analytical solution for a straight fin with an insulated tip approximation: $\eta_f = \frac{\tanh(mL_c)}{mL_c}$. This fraction represents the actual heat transfer

from the fin divided by the ideal heat transfer if the entire fin remained at the root (tube wall) temperature.

Surface efficiency calculated as $\eta_0 = 1 - \frac{A_{secondary}}{A_{primary} + A_{secondary}} (1 - \eta_f)$. This creates a weighted average of the two distinct surfaces. The bare tube wall ($A_{primary}$) is treated as 100% efficient because it is in direct proximity to the coolant, while the fin surface ($A_{secondary}$) is penalized by its thermodynamic efficiency η_f .

```
%A_fins calculation
A_primary = 2*d*W*(H/(a_tube+f_height)); % tube and air contact surface area
A_secondary = 2*f_height*d*fpi*W*(H/(a_tube+f_height))/0.0254; % fin and air contact surface area
k_fin = 237; %thermal conductivity of aluminium (fin material)
% fin efficiency
m = sqrt(2*h_air/(k_fin*f_thickness));
L_c = props.f_height/2 + props.f_thickness/2;
eta_f = tanh(m*L_c)/(m*L_c);
eta_0 = 1-(A_secondary/(A_primary+A_secondary))*(1-eta_f); %surface efficiency
A_fins = eta_0*(A_primary+A_secondary);
```

While fin efficiency measures how well a fin performs against its own ideal, fin effectiveness ϵ_f asks a different question: is the fin worth adding at all, compared to leaving that surface bare? It is defined as the ratio of actual fin heat transfer to the heat transfer that same base footprint would provide with no fin present, $\epsilon_f = \eta_f (A_{secondary}/A_{c_b})$, where A_{c_b} is the total root area the fins occupy on the tube wall. As a rule of thumb, $\epsilon_f > 2$ justifies the added weight and pressure-drop cost of finning a surface; automotive radiator fins typically return values well into the tens, since the thin, tall fin geometry gives a large area ratio even after the efficiency penalty is applied.

```
A_c_b = f_thickness*d*n_tubes*2*fpi*W/0.0254;
epsilon_f = eta_f*(A_secondary/A_c_b);
epsilon_f_mat(i,j) = epsilon_f;
```

FPI	FinEffectiveness
8	93.165
9	92.944
10	92.733
11	92.531
12	92.337
13	92.151
14	91.972
15	91.798
16	91.63
17	91.466
18	91.307
19	91.152
20	91.001
21	90.853
22	90.709
23	90.567
24	90.429
25	90.293

Figure 2: Fin effectiveness values for varying values of `fpi`

2.9 NTU, ϵ , and Predicted Heat Rejection

This is computed across the full `A_face` $\times$ `fpi` grid, producing a 2D map of predicted heat rejection.

```
NTU = (U*A_fins)/C_min; %number of transfer units
epsilon = 1-exp((NTU^0.22/C_star)*(exp(-C_star*NTU^0.78)-1));
Q_predicted(i,j) = epsilon*C_min*(T_coolant_in-T_air_in);
```

2.10 Extracting the Design Point

A filled contour plot of `Q_predicted` over the `fpi/A_face` grid is generated, with the `Q_target` contour and a `Q_target` $\times$ 1.1 (10% safety margin) contour overlaid. This is a design-margin plot: any point above/right of the `Q_target` line is a geometry that clears the heat rejection requirement. The exact minimum frontal area at a chosen `fpi` is then extracted numerically by pulling the target-Q contour line coordinates (`contourc`) and interpolating along it (`interp1`) at the desired `fpi`, and the required radiator width is derived from `A_face/H` and plotted the same way.

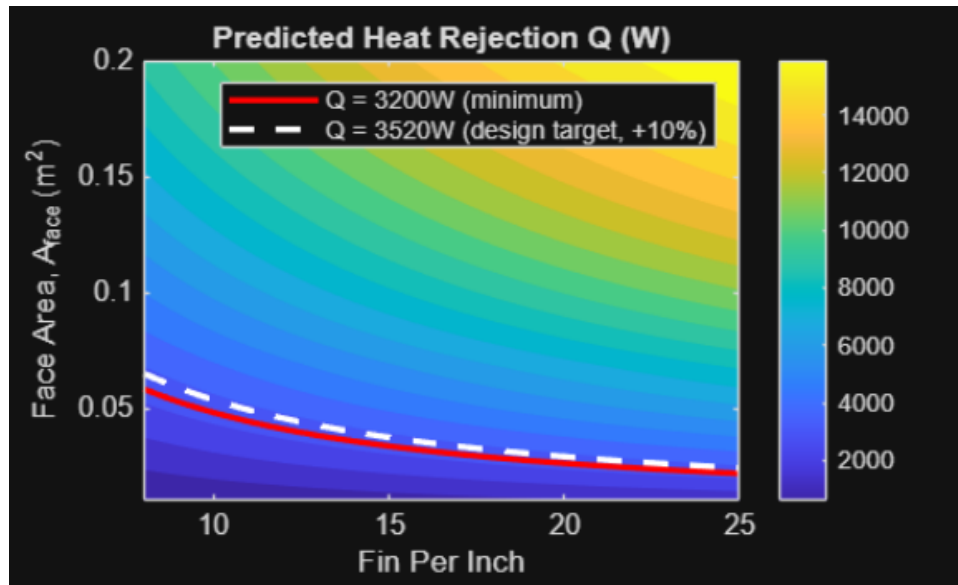


Figure 3: Predicted heat rejection $Q(W)$ over the fpi / A_{face} design grid, with the Q_{target} and +10% margin contours overlaid.

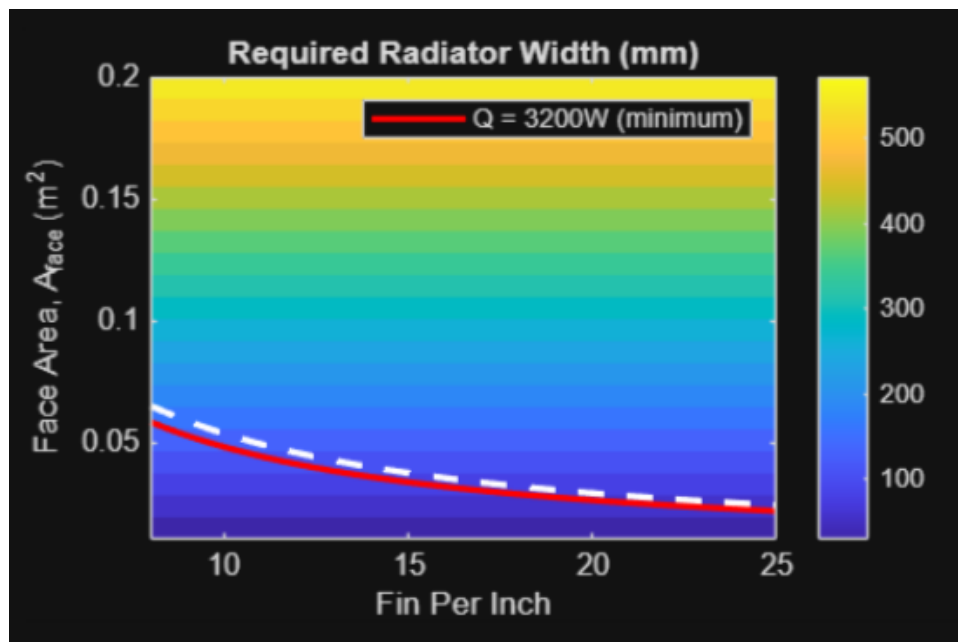


Figure 4: Required radiator width (mm) over the same design grid, with the Q_{target} contour overlaid.

2.11 Water-Side Pressure Drop

At the extracted design point, a Darcy friction factor for laminar flow ($f = 64/Re$) is used to estimate the pressure drop across the radiator core. This is a first-pass check only (a more detailed procedure is planned once pump selection is underway); it covers the water side only, **air-side pressure drop and fan/pumping power (if used) are not modelled.**

At fpi = 18, minimum A_{face} ($Q=3200W$) = 0.0286 m^2
 At chosen fpi and A_{face_min} ($H=350\text{mm}$), Pressure drop = 7.17 Pa

At $f_{pi} = 18$, design target A_{face} ($Q=3520W$, $+10\%$) = 0.0318 m^2
 At chosen f_{pi} and A_{face_min} ($H=350\text{mm}$), Pressure drop = 7.97 Pa

Flag: this is only the pressure drop across the radiator core itself. It does not account for the rest of the cooling circuit (motor, motor controller, pump, and connecting pipes), so it should not be read as the total system pressure drop the pump needs to overcome.

2.12 Sensitivity Studies

Two one-way sensitivity sweeps are run, holding the chosen design point's f_{pi} and A_{face} fixed:

- **Height (H) sweep, 300–400 mm:** shows that at fixed f_{pi} and frontal area, varying height has a negligible effect on predicted Q. This justifies treating H as a fixed 350 mm parameter in the main sweep rather than a third swept dimension. (The zig-zag pattern is due to the number of tubes in the radiator being a step function, it only increases after a certain height increase, and while height increases for a fixed area, width decreases, which further reduces both $A_{primary}$ and $A_{secondary}$.)

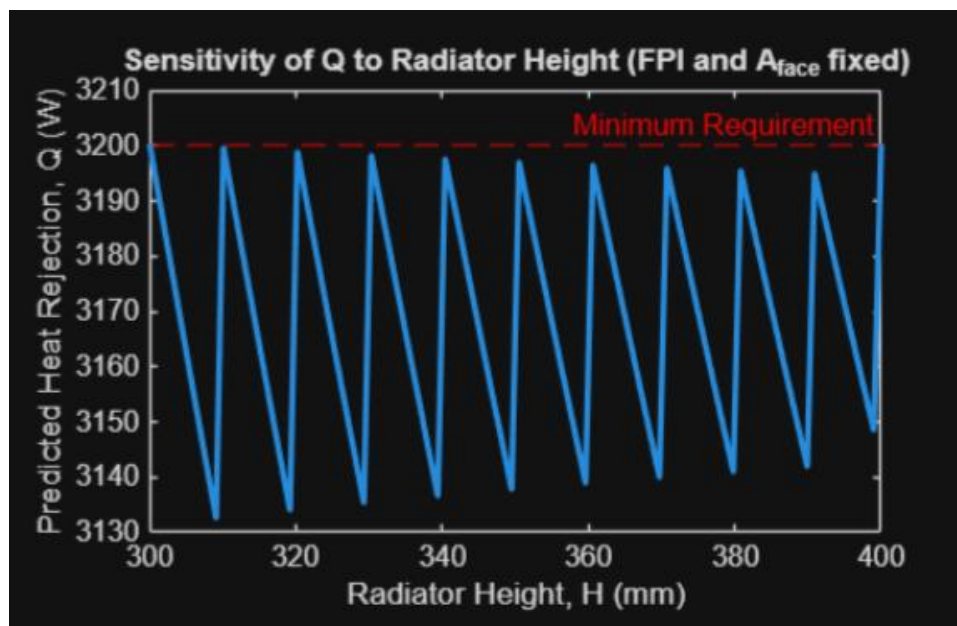


Figure 5: Sensitivity of predicted Q to radiator height H, with f_{pi} and A_{face} fixed at the design point.

- **Coolant flow rate sweep, 3 - 24L/min:** shows Q rising quickly at low flow rate (where $C_{min} = C_{water}$, i.e. the coolant side is the bottleneck) before flattening out once flow rate crosses the point where C_{min} switches to the air side ($C_{min} = C_{air}$). Past that crossover, more coolant flow barely changes Q because heat rejection is now bottlenecked by the air side. This crossover flow rate is calculated explicitly and marked on the plot alongside the current pump's rated flow.

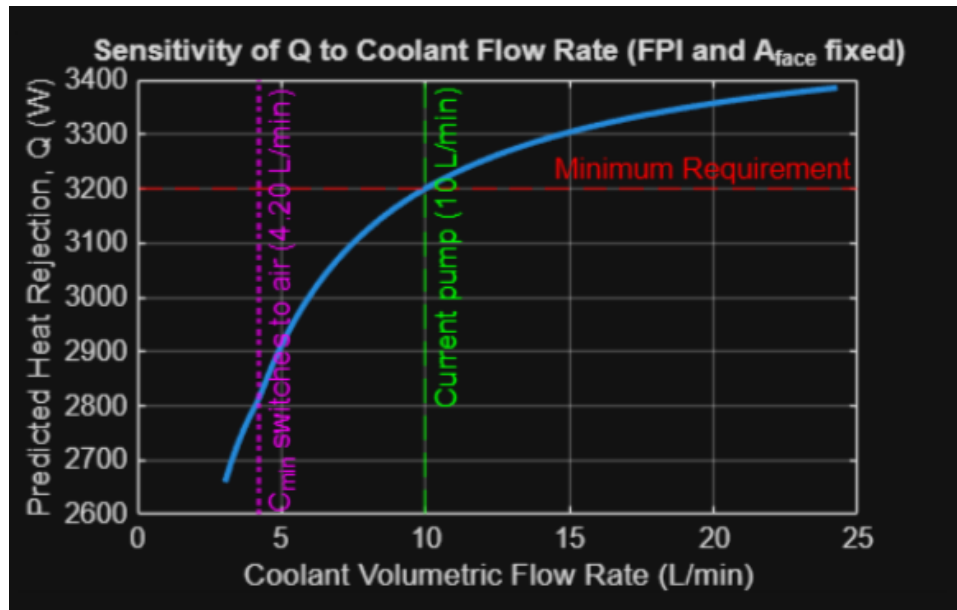


Figure 6: Sensitivity of predicted Q to coolant mass flow rate, showing the C_{min} crossover from water-side- to air-side-limited.

2.13 A_{face} vs. FPI Summary Table

A final summary loop prints the minimum required frontal area (and corresponding width) at each swept fpi value, giving a quick reference table for trading off fin density against radiator size.

FPI	A_{face} (m ²)	Width (mm)
8	0.0583	145.7
9	0.0526	131.6
10	0.0480	120.0
11	0.0441	110.4
12	0.0409	102.2
13	0.0381	95.1
14	0.0356	89.1
15	0.0335	83.8
16	0.0317	79.2
17	0.0300	75.1
18	0.0286	71.4
19	0.0273	68.2
20	0.0261	65.3
21	0.0250	62.6

FPI	A_face (m ²)	Width (mm)
22	0.0241	60.2
23	0.0232	58.1
24	0.0224	56.1
25	0.0217	54.3

3. Transient Models (radiator_transient_fmot.mlx / radiator_transient_emrax.mlx)

The steady-state script sizes the radiator against a single conservative worst-case operating point. The transient model instead replays a real endurance-run data set through the chosen geometry and sweeps the coolant temperature with time, to check whether the design survives the actual, time-varying accel/brake/idle profile of an event rather than just the theoretical worst case. radiator_transient_fmot.mlx and radiator_transient_emrax.mlx are structurally identical; the only difference between them is the motor + controller efficiency map, since each motor has its own datasheet efficiency contours and different max input values of torque and power possible.

3.1 Extracting Data as Input

Data is read from a logged endurance run (FSPT_endurance.xlsx, a placeholder filename for whatever file is being used to record data):

```
clear; clc; close all;
dataTable = readtable('FSPT_endurance.xlsx'); %placeholder name
t_data = dataTable.Timestamp/1000000; %(s)
torque_data_excel = dataTable.Torque_Right;
torque_data = torque_data_excel*(67.88/32767); %(Nm)
tsv = dataTable.TSV; % Tractive System Current
tsc = dataTable.TSC; % Tractive System Voltage
power_data = tsv .* tsc; %(W)
rpm_data = dataTable.RPM_RR;
```

3.2 Data Cleaning: Zero-RPM Filtering and Time Shifting

Samples where the motor is not spinning are dropped, and the remaining timestamps are re-accumulated so the simulation timeline is continuous (no gaps left behind by the removed samples). This keeps the model focused on periods where the motor is actually generating heat and moving the car.

```
valid_idx = rpm_data ~= 0;
dt_orig = [0; diff(t_data)];
t_data = cumsum(dt_orig(valid_idx));
```

```
torque_data = torque_data(valid_idx);  
power_data = power_data(valid_idx);  
rpm_data = rpm_data(valid_idx);
```

Flag: this compresses out real elapsed time during idle/stationary periods. Since $Q_{gen} \approx 0$ at zero RPM this is a reasonable approximation for heat generation, but it does mean the simulated timeline no longer matches the real event duration — any heat soak or radiator cooling that would have continued to occur during idle (e.g. between endurance stints) is not represented.

3.3 Motor + Controller Efficiency Map

RPM–torque–efficiency data points from the motor + controller combined efficiency map (datasheet contour lines, hand-extracted into `rpm_pts/torque_pts/eff_pts` triplets) are interpolated with a scattered interpolant:

```
F = scatteredInterpolant(rpm_pts(:), torque_pts(:), eff_pts(:), 'linear',  
    'nearest');
```

At each timestep, efficiency is looked up at the current (RPM, torque) operating point, clamped to $[0,1]$, and used to convert electrical input power into dissipated heat:

```
% --- heat generated ---  
eff = F(rpm_data(k), torque_data(k));  
P_check = torque_data(k)*rpm_data(k)*2*pi/60; % (W)  
if torque_data(k) > 31.5 || power_data(k) > 35000  
    warning('t=%.2fs: T=%.1f Nm, RPM=%.0f, Logged P=%.0f W, computed P=%.0f W',  
        ...  
        t_data(k), torque_data(k), rpm_data(k), power_data(k), P_check);  
end  
eff = min(max(eff, 0), 1); % efficiency between 0-1 (decimal)  
Q_gen = power_data(k)*(1-eff); %both motor and motor controller  
Q_gen_history(k) = Q_gen;
```

This is a **combined** motor+controller loss figure straight from the map, in contrast to the steady script's separate motor-loss + fixed-750 W-MC accounting.

Flag: lines 192–194 above are warning messages added when the data was outputting more torque or power than the motor can produce. The current `FSPT_endurance.xlsx` data is from the older motors, which have a higher limit than the newer motors - running this code with that data will spam warning messages until it is replaced with F-MOT telemetry.

3.4 Initial Parameters (Flagged for Iteration)

The geometry values are meant to be carried over from whatever design point was extracted in the steady script. `gear_ratio` and `ram_air_factor` are marked as placeholder values, pending whatever values are decided by the drivetrain/aero team in the future.

```
% MAKE CHANGES HERE
A_face_fixed = 0.0286;
fpi_fixed = 18;
H_fixed = 0.35; %(m)
d = 0.032; %(m), depth of radiator
W = A_face_fixed/H_fixed; %(m), width of radiator
mdot_coolant = 10; %(L/min), per side, fixed before the loop
T_air_in = 25; %(degC)
T_coolant_initial = 25; %(degC)
r_wheel = 0.11; %(m)
gear_ratio = 4.5;
ram_air_factor = 0.7; %placeholder value
```

3.5 Deriving Air Velocity from Wheel Kinematics

Rather than assuming a single fixed `v_air` like the steady script, the transient model derives it from the car's actual speed at each timestep, via motor RPM and the drivetrain ratio:

```
% --- vehicle & air speed ---
omega_motor = rpm_data(k)*2*pi/60; % (rad/s)
omega_wheel = omega_motor/gear_ratio;
v_car = r_wheel*omega_wheel; % (m/s)
v_air = ram_air_factor*v_car; % (m/s)
v_air = max(v_air, 0.5); % floor to avoid v_air = 0 (stalled/idle) blowing up
h_air
v_air_history(k) = v_air;
```

Flag: `ram_air_factor` depends on the placement of the radiator on the car, which is to be decided by the aero team, but a position can't be decided unless we provide them with the required air mass flow rate through the radiator, which itself depends on `v_air` into the radiator. Future discussion required on this issue.

3.6 Q_epsilonNTU Function

The steady-state script's inline heat-rejection calculation is refactored into a standalone function so it can be called once per timestep:

```
function [Q_rejected, water_state] = Q_epsilonNTU(T_coolant_in, v_air,
mdot_coolant_kgs, A_face, fpi, H, props, water_state)
```

Two performance considerations are built in:

- **props struct:** all the fixed geometry and air-property constants are packed into a struct once, outside the time loop, instead of being recomputed every timestep.
- **water_state:** water properties (k , μ , Pr) are only re-queried from CoolProp when the coolant temperature has moved by $\geq 1^\circ\text{C}$ since the last lookup, rather than on every single timestep (to reduce computational time for long python calls required by CoolProp). Since these properties change slowly with temperature, this significantly cuts down on repeated CoolProp/Python calls over long-running data without changing accuracy significantly. **Air properties, by contrast, remain fixed at the 25°C reference value for the entire run.**

3.7 Time-Marching Loop (Lumped Capacitance, Explicit Euler)

The coolant in the radiator core is treated as a single lumped thermal mass:

```
m_water = props.rho_water*props.A_tube*W*n_tubes; % mass of water held in the  
radiator core
```

Flag: this is the mass of coolant in the radiator core only, not the full coolant loop volume (motor jacket, motor controller, pipes, pump). The lumped-capacitance model is only as good as this thermal mass estimate; using only the radiator's own volume likely understates the total thermal mass of the loop, which would make the simulated temperature respond faster (both heating and cooling) than the real system.

Each step is a first-order forward (explicit) Euler update:

```
T_new = T_old + dt*(Q_gen - Q_rejected)/(m_water*cp_water);
```

The full temperature history is stored for every timestep (not just the final value), so the whole thermal history can be inspected. Timesteps where the 70°C limit is exceeded are logged (first-exceedance time, peak temperature) without breaking the loop, so the full duration of any exceedance is captured rather than just its first occurrence.

The 70°C upper limit is chosen as the closest-to-safe operating temperature of the coolant that the motor controller (being a highly temperature-sensitive device) can feasibly operate on.

3.8 Outputs

- Coolant temperature vs. time, with a 70°C limit reference line.
- Peak temperature and time of first limit violation (or confirmation the run stayed under limit).
- Q_{gen} vs. Q_{rejected} over time: visualizes where in the run the radiator is keeping up vs. falling behind.
- A post-warm-up minimum temperature check (looking only after a threshold time, e.g. 100 s in) to see where the system settles once initial transients have died out.

4. F-MOT vs. EMRAX: What's Different

The two transient scripts are near-identical: same data pipeline, same `Q_epsilonNTU` function, same time loop, same outputs. The **only** difference between them is the efficiency map (`rpm_pts/torque_pts/eff_pts`) used to build the `scatteredInterpolant`, since each motor + controller combination has its own datasheet efficiency contours — and the maximum possible torque and power values used for the warning message.

5. Validation Suite (`radiator_transient_test.mlx`)

To verify proper working of the transient model, it was necessary to check that if a constant load (RPM and torque) is given as input, the coolant temperature converges to a value equal to the independently solved steady-state temperature.

5.1 Independent Steady-State Solver

Rather than trusting the time-marching result on its own, each test case is cross-checked against a steady-state coolant temperature found by root-finding, completely independently of the time-stepping loop. This solves $Q_{gen} = Q_{rejected}(T_{ss})$ directly: the temperature at which heat generated exactly balances heat rejected, using bisection between the ambient temperature and 150°C. If the sign of the residual doesn't change across that bracket, no steady-state solution exists in range.

```
% independent steady-state solve:  $R(T) = Q_{gen} - Q_{rejected}(T)$ 
% bisection method
residual = @(T) Q_gen-Q_epsilonNTU(T, c.v_air, mdot_coolant_kgs, A_face_fixed,
fpi_fixed, H_fixed, props, []);
T_lo = T_air_in+0.01; % bounds for temp search range, temp cant be below t_air
T_hi = 150; % 150 taken as an upper bound as coolant never reaches this temp
R_lo = residual(T_lo);
R_hi = residual(T_hi);

if Q_gen == 0 % no heat Load means the coolant just relaxes to ambient air temp
    T_ss = T_air_in;
    status = 'converged';
    diff_val = T_final_sim-T_ss;
else
    T_lo = T_air_in+0.01;
    T_hi = 150;
    R_lo = residual(T_lo);
    R_hi = residual(T_hi);

    if sign(R_lo) == sign(R_hi)
        T_ss = NaN;
        status = 'NO STEADY SOLUTION (overload) - check T_final_sim is
monotonically rising';
        diff_val = NaN;
```

```

else
    T_ss = fzero(residual, [T_lo T_hi]);
    status = 'converged';
    diff_val = T_final_sim-T_ss;
end
end

```

5.2 Test Case Matrix

Seven constant-condition cases (held RPM/torque/air velocity, run out to a fixed duration) probe different aspects of the model's correctness:

Case	Conditions	What it checks
A: Zero load	RPM=5000, torque=0	With no heat generated, coolant temperature should relax to ambient air temperature.
B: Nominal design point	RPM=8000, torque=20 Nm, $v_{air}=10$	A representative mid-map operating point, approached from a cold start ($T_{initial} = 25^{\circ}\text{C}$).
C: Overload / stall	RPM=2000, torque=26 Nm (near max), v_{air} floored at 0.5	Heat generated exceeds the maximum heat the radiator can reject at that air velocity, no steady-state solution exists (in the 70°C limit). Confirms the model correctly identifies a genuine thermal runaway condition rather than silently producing a wrong equilibrium.
D: Nominal, approached from above	Same as B, but $T_{initial} = 80^{\circ}\text{C}$	Confirms the same equilibrium temperature is reached whether approached from below (case B) or above (case D).
E1 / E2 / E3: Air velocity sweep	Same nominal Q_{gen} as B, $v_{air} = 5, 15, 30$ respectively	Confirms monotonic behaviour: higher air velocity $\rightarrow$ more heat rejected $\rightarrow$ lower equilibrium coolant temperature ($E1 > E2 > E3$ in final temperature).

5.3 Time Constant and t_{99}

For each case, a thermal time constant is estimated from the final-state conductance:

```
tau_est = m_water*cp_water/UA_final;
```

and t_{99} : the time to close 99% of the gap between the initial and final (independently solved) steady-state temperature, found by linear interpolation between the two simulated samples that bracket the 99% point. For a first-order lumped system, $t_{99} \approx 4.6 \cdot \tau_{est}$ (since $\ln(100) \approx 4.6$); this relationship is checked directly in the printed summary as a sanity check that the simulation is reproducing expected first-order step-response behaviour when $dt \ll \tau_{est}$.

5.4 Summary Table and Pass/Fail Checks

The script prints a table of Q_{gen} , simulated final temperature, independently-solved steady-state temperature, their difference, τ_{est} , t_{99} , and status for all seven cases, followed by explicit written checks to manually verify against:

- $|diff|$ should be small (sub-0.1°C) for cases A, B, D, E1–E3 when $dt \ll \tau_{est}$.
- Case C should report converging temperature higher than feasible operating range.
- $E1 > E2 > E3$ in final temperature.
- Case D should converge to the same steady-state temperature as case B.
- $t_{99} \approx 4.6 \cdot \tau_{est}$ across the cases with a defined steady state.

Only once these checks pass is the same $Q_{epsilonNTU}/\text{time-sweep}$ combination considered trustworthy enough to run against real data in the F-MOT/EMRAX scripts.

6. Known Assumptions and Simplifications

Consolidated from all four scripts, roughly in order of how likely they are to matter:

- **Thermal mass scope:** m_{water} in the transient models is the coolant mass held in the radiator core only, not the full coolant loop (motor, controller, hoses, pump). Likely understates total system thermal mass.
- **Ram-air factor:** $ram_air_factor = 0.7$ is a placeholder constant relating car speed to actual air velocity through the core; not yet derived from aero/CFD data.
- **Colburn j-factor correlation:** $J = 0.174/Re^{0.383}$ is inherited from the prior code base; its source and applicability to this specific fin geometry has not yet been independently confirmed against Kays & London reference data.
- **Motor controller loss accounting:** the steady script uses a fixed 750 W MC dissipation figure on top of motor loss; the transient scripts pull a combined motor+MC loss straight from the efficiency map.
- **Air-side properties fixed:** air density, viscosity, conductivity, and Prandtl number are all evaluated once at 25°C and held constant for the entire run; only water properties are updated as temperature changes.
- **Air-side pressure drop / fan power:** not modelled, as fan usage for the radiator is not yet decided. Only water-side pressure drop (laminar Darcy friction factor) is estimated, and only at the design point in the steady script.
- **Fin/tube bond resistance:** assumed negligible, pending manufacturing/bonding process confirmation.
- **Height (H) sensitivity:** shown to be small over the 300–400 mm design range, which is the justification for holding H fixed at 350 mm rather than sweeping it as a third design variable.
- **Zero-RPM filtering:** dropping and time-compressing idle samples is reasonable for heat generation (which is ≈ 0 at idle), but means the simulated timeline doesn't match real elapsed event time, and doesn't explicitly model any cooling/heat soak that would occur during real idle periods.

- **Single-event scope:** all validation so far is against a single endurance run or synthetic constant-condition case. Thermal ratcheting across multiple repeated endurance stints (i.e. whether the coolant fully recovers between stints, or slowly climbs run-over-run) is not yet modelled or tested.

7. Suggested Workflow

1. Run **radiator_steady.mlx** to sweep FPI/frontal area and extract a minimum viable geometry (A_{face} , fpi) against the fixed target heat load, at a fixed worst-case operating point.
2. Feed that geometry into the **% MAKE CHANGES HERE** block of the transient scripts.
3. Run **radiator_transient_test.mlx** first, on synthetic constant-condition cases, to confirm the time-marching implementation numerically agrees with an independently solved steady-state temperature before trusting it on real data.
4. Run **radiator_transient_fmot.mlx** / **radiator_transient_emrax.mlx** against real endurance-run telemetry to check whether the chosen geometry keeps coolant temperature under the 70°C limit across an entire representative run, not just at one static worst-case point.